SESSION 3 LAB PROGRAMS

Name: Milan. A
Register Number: 192424377
Course: Design Analysis Of Algorithm
Course code: CSA0604

1. Given an m x n grid and a ball at a starting cell, find the number of ways to move the ball out of the grid boundary in exactly N steps.

Example:

Input: m=2,n=2,N=2,i=0,j=0 Output: 6

Input: m=1,n=3,N=3,i=0,j=1 Output: 12

CODE:

```
#include <stdio.h>
#include <string.h>

int main() {
    int m, n, N, si, sj, k, i, j, d, ni, nj;
    int dx[4] = {-1, 1, 0, 0}, dy[4] = {0, 0, -1, 1};
    long long dp[50][50], nd[50][50], count = 0;

    printf("Enter m, n: ");
    scanf("%d %d", &m, &n);
    printf("Enter N: ");
    scanf("%d", &N);
    printf("Enter starting cell i, j: ");
    scanf("%d %d", &si, &sj);

    memset(dp, 0, sizeof(dp));
    dp[si][sj] = 1;                /* start cell */

    for (k = 1; k <= N; k++) {
        memset(nd, 0, sizeof(nd));
        for (i = 0; i < m; i++)
            for (j = 0; j < n; j++) {
                if (dp[i][j] == 0) continue;
                for (d = 0; d < 4; d++) {
                    ni = i + dx[d];
                    nj = j + dy[d];
                    if (ni < 0 || ni >= m || nj < 0 || nj >= n)
                        count += dp[i][j];       /* leaves grid */
                    else
                        nd[ni][nj] += dp[i][j];  /* stays inside */
                }
            }
        memcpy(dp, nd, sizeof(dp));
```

```
}
printf("Number of ways: %lld\n", count);
return 0;
```

}

OUTPUT:

```
Enter m, n: 2 2
Enter N: 2
Enter starting cell i, j: 0 0
Number of ways: 6
```

```
Enter m, n: 1 3
Enter N: 3
Enter starting cell i, j: 0 1
Number of ways: 12
```

Time Complexity: O(N × m × n) | Space Complexity: O(m × n)

2. You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses at this place are arranged in a circle. That means the first house is the neighbor of the last one. Meanwhile, adjacent houses have security systems connected, and it will automatically contact the police if two adjacent houses were broken into on the same night.

Examples:

Input : nums = [2, 3, 2]

Output : The maximum money you can rob without alerting the police is 3(robbing house 1).

(ii) Input : nums = [1, 2, 3, 1]

Output : The maximum money you can rob without alerting the police is 4 (robbing house 1 and house 3).

CODE:

```
#include <stdio.h>

/* maximum money from houses a[lo..hi] in a straight line */
int robLine(int a[], int lo, int hi) {
   int prev = 0, curr = 0, t, i;
   for (i = lo; i <= hi; i++) {
      t = (prev + a[i] > curr) ? prev + a[i] : curr;
      prev = curr;
      curr = t;
```

```
    }
    return curr;
}

int main() {
    int n, i, a[100], r1, r2, ans;
    printf("Enter number of houses: ");
    scanf("%d", &n);
    printf("Enter the money in each house: ");
    for (i = 0; i < n; i++) scanf("%d", &a[i]);

    if  (n  ==  1)
        ans = a[0];
    else {
        r1 = robLine(a, 0, n - 2);   /* skip the last house  */
        r2 = robLine(a, 1, n - 1);   /* skip the first house */
        ans = (r1 > r2) ? r1 : r2;
    }
    printf("The maximum money you can rob without alerting the police is %d\n", ans);
    return 0;
}
```

OUTPUT:

```
Enter number of houses: 3
Enter the money in each house: 2 3 2
The maximum money you can rob without alerting the police is 3
```

```
Enter number of houses: 4
Enter the money in each house: 1 2 3 1
The maximum money you can rob without alerting the police is 4
```

Time Complexity: O(n) | Space Complexity: O(1)

3. You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Examples:

Input: n=4 Output: 5

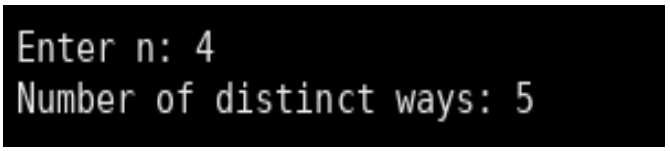
Input: n=3 Output: 3

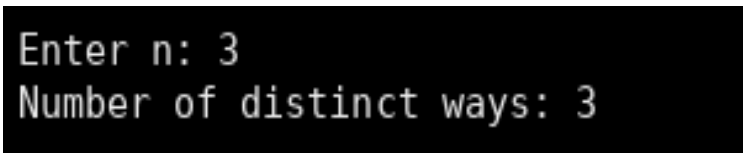
CODE:

```
#include <stdio.h>

int main() {
    int n, i;
    long long dp[50];
    printf("Enter n: ");
    scanf("%d", &n);
    dp[0] = 1;
    dp[1] = 1;
    for (i = 2; i <= n; i++)
        dp[i] = dp[i - 1] + dp[i - 2];   /* last step 1 or 2 */
    printf("Number of distinct ways: %lld\n", dp[n]);
    return 0;
}
```

OUTPUT:





Time Complexity: O(n) | Space Complexity: O(n)

4. A robot is located at the top-left corner of a m×n grid .The robot can only move either down or right at any point in time. The robot is trying to reach the bottom-right corner of the grid. How many possible unique paths are there?

Examples:

Input: m=7,n=3 Output: 28

Input: m=3,n=2 Output: 3

CODE:

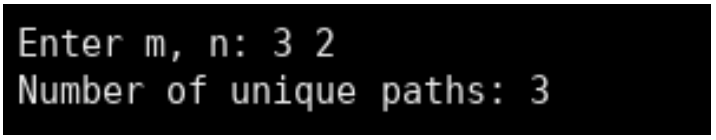
```
#include <stdio.h>

int main() {
    int m, n, i, j;
```

```
    long long dp[50][50];
    printf("Enter m, n: ");
    scanf("%d %d", &m, &n);
    for (i = 0; i < m; i++)
        for (j = 0; j < n; j++) {
            if (i == 0 || j == 0)
                dp[i][j] = 1;
            else
                dp[i][j] = dp[i - 1][j] + dp[i][j - 1];
        }
    printf("Number of unique paths: %lld\n", dp[m - 1][n - 1]);
    return 0;
}
```

OUTPUT:





Time Complexity: O(m × n) | Space Complexity: O(m × n)